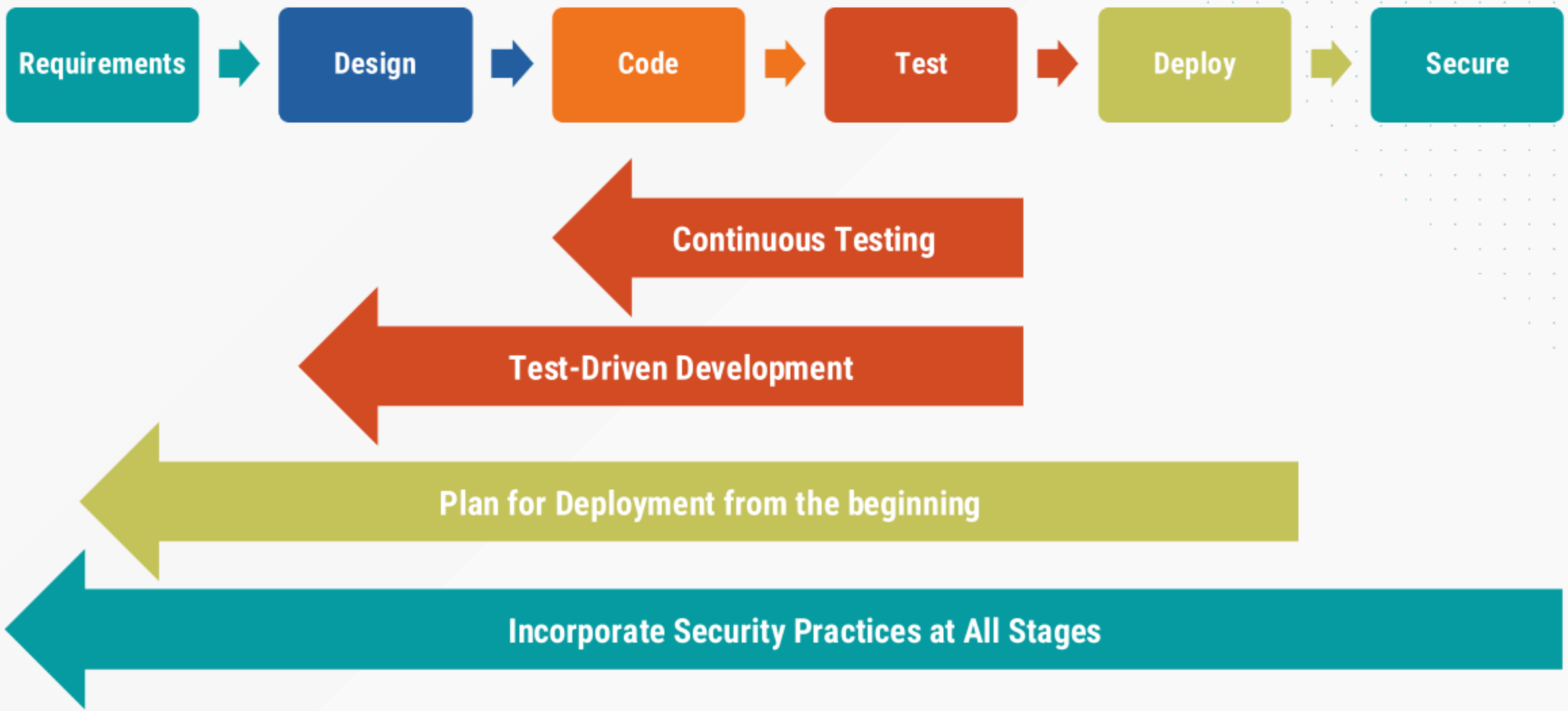
Monitoring

What's Going On?

Our Project

1. Managed with git
2. Running Docker containers
3. Orchestrated with Kubernetes
4. Tested in Gitlab
5. Deployed with ArgoCD
6. **Monitored with Prometheus and Grafana**

DevSecOps: *Shift Left*



Importance of Monitoring

Even when a system works in testing, the real world can change

- User behavior changes
- External services change their behavior
- External services break or disappear
- Hardware fails
- Disk space is exhausted

We need to know what is actually happening

Prometheus and Grafana

Prometheus is a time-series database

- Periodically injests data from a source
- Stores the data
- Runs queries on this data

Grafana is an analytics and visualization web application

- Queries a wide variety of databases (including Prometheus)
- Displays selected metrics in dashboards

Note: *minikube* Won't Run This Well

Our minikube clusters don't have enough resources to run all of these components well

I will demonstrate on a real Kubernetes cluster

You will have access to this cluster during the capstone project

Installing a Monitoring Stack

Several components will need to be installed and configured to work together

The kube-prometheus project collects and configures these components into a working solution

It can be installed from a collection of YAML manifests or a community helm chart

Installing From the Helm Chart

```
helm install prometheus \
  oci://ghcr.io/prometheus-community/charts/kube-prometheus-stack \
  --namespace monitoring --create-namespace
```

The `oci://` scheme indicates that the chart is stored in an OCI-compliant registry

→ "Open Container Initiative" is the vendor-neutral way to say "Docker"

This will take a minute or two to complete

Activity: What Got Installed?

How can we check what got installed?

- Pods
- Deployments
- Services
- *others?*

Prometheus Sits at the Heart

Prometheus is running in the *prometheus-kube-prometheus-prometheus* service

It exposes a web UI on port 9090:

```
kubectl port-forward -n monitoring \
  service/prometheus-kube-prometheus-prometheus 9090
```

Prometheus Web UI

The web UI lets us query the time-series database

One metrics being tracked is *node_memory_MemAvailable_bytes*
→ Enter this in the *Expression* box and press *Execute*

The *Table* interface shows the current value of this metric

There are three numbers, one for each node in our cluster
→ Note the *instance* property is different for each

The *Evaluation time* widget shows values at different points in time

Graphing a Time Series

Switch to the *Graph* interface to see these values change over time

Since Prometheus has just started up, there is rather limited range of times available right now

Calculated Values

Change the *Expression* to

```
node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes
```

Now the plot shows the fraction of memory free on each node

A wide variety of operations is available

Filtering Series

Change the expression to `container_cpu_usage_seconds_total`
This shows the total number of CPU-seconds used by each container

Each series has a number of labels. We can filter on those labels by including an expression in curly braces:

```
container_cpu_usage_seconds_total{namespace='dso-project',
  container='web-container'}
```

⇒ Shows only containers named *web-container* in *dso-project* namespace

Examining Rates

The rate at which the total CPU usage is changing tells us how much CPU is being used at any point in time

```
rate(container_cpu_usage_seconds_total{namespace='dso-project',
  container='web-container'}[1m])
```

`rate` is a function provided by Prometheus

`[1m]` indicates the rate is calculated over one-minute segments

Six Installed Workloads

- *prometheus-prometheus-kube-prometheus-prometheus*
  - StatefulSet running Prometheus
  - Stores the time-series database
  - Ingests data from various sources
  - Runs queries on the database
- *prometheus-kube-prometheus-operator*
  - Deployment running the Prometheus Operator
  - Watches for custom resources representing Prometheus configs
  - Updates Prometheus when custom resources change

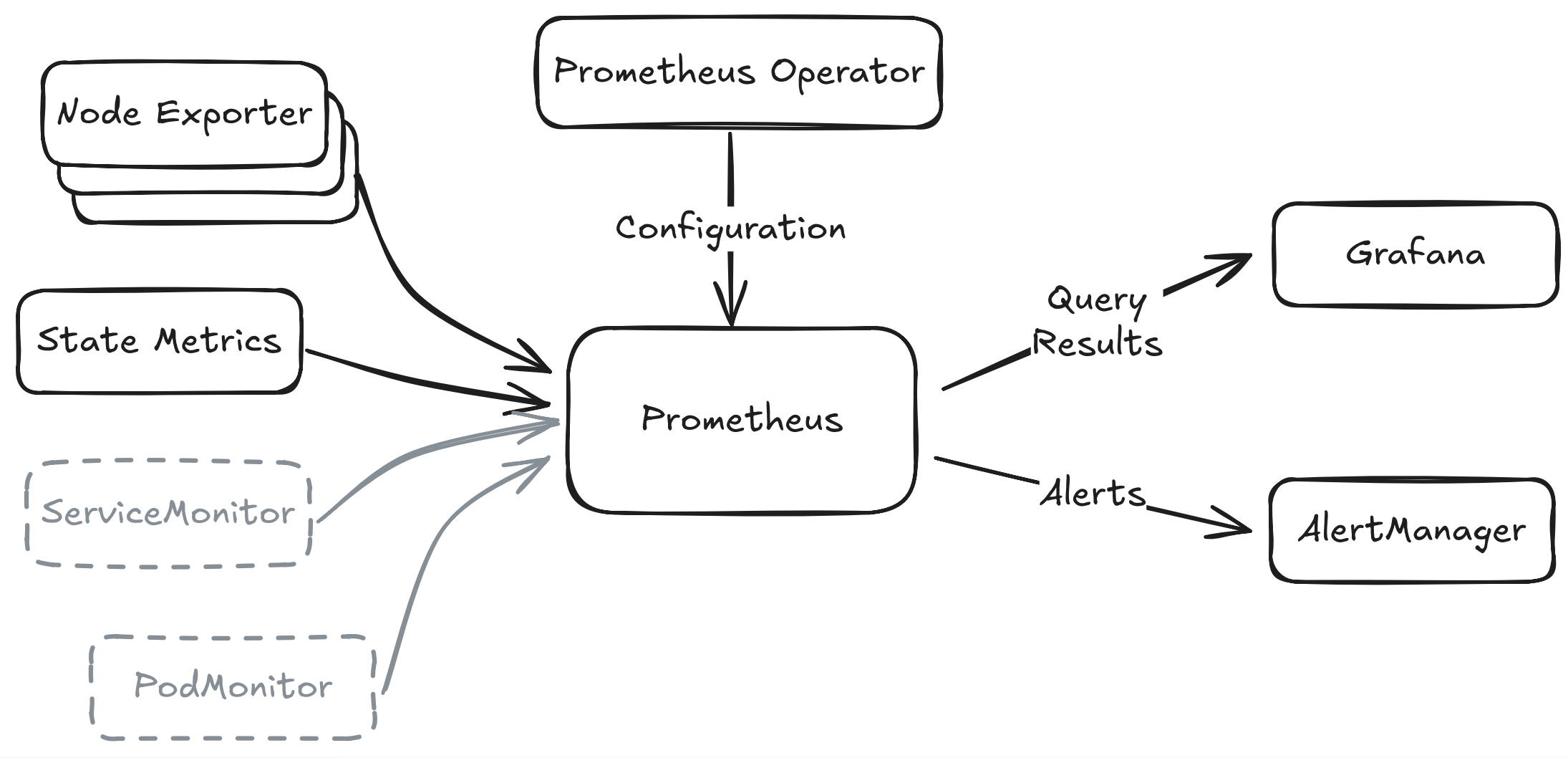
Six Installed Workloads

- *prometheus-prometheus-node-exporter*
  - DaemonSet runs one pod per node
  - Gathers node metrics for Prometheus to collect
- *prometheus-kube-state-metrics*
  - Deployment of pod to talk to K8s API server
  - Gathers metrics on cluster state (pods, nodes, PVCs, *etc*)

Six Installed Workloads

- *alertmanager-prometheus-kube-prometheus-alertmanager*
  - StatefulSet running the Prometheus AlertManager
  - Receives alerts from Prometheus
  - Deduplicates, groups, silences alerts as appropriate
  - Routes alters to specified receivers
- *prometheus-grafana*
  - Deployment of Grafana
  - Web UI for exploration and dashboarding

Architecture



Running Services Outside the Cluster

Prometheus

✅ Survives cluster crash

🔴 Need to route all metrics providers

🔴 Service account outside cluster

🔴 Can't configure with Operator

- Less common

Grafana

✅ Connect to many sources

✅ Managed options

✅ Easier persistence

🔴 Need to expose Prometheus

🔴 Can't configure with ConfigMaps

- Somewhat common

Visualizing Data with Grafana

Grafana sits behind a login. Get the default password:

```
kubectl get secret -n monitoring prometheus-grafana \
  -o jsonpath="{.data.admin-password}" | base64 -d; echo
```

Set up port forwarding:

```
kubectl port-forward -n monitoring service/prometheus-grafana 8088:80
```

Login in with username *admin* and password from above

Explore Prometheus Data in Grafana

Click *Explore* in the left sidebar

In *Metric*, enter `container_cpu_usage_seconds_total`

Click *Run query* to see graph and raw results

Add *Label filters*

- Set *namespace* to `dso-project`
- Set *container* to `web-container`

Explore Data Rate

There should be a hint saying *add rate*
Click it and select a *range* of `1m`

Re-run the query to see the updated plot

Note that the Prometheus query is displayed
$\Rightarrow$ Can be edited directly by switching from *Builder* to *Code*

Create a Dashboard

1. Click *Add* ❭ *Add to Dashboard*
2. Select *New dashboard*, then click *Open dashboard*
3. Click *Save*
4. Enter a title and click *Save*

Edit the Dashboard

1. Click *Edit*
2. Below the exisitng panel, click *Add panel*
3. Click *Configure visualization*
4. Switch to *Code* mode and enter

   ```
   node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes
   ```

5. Click *Run queries* to see a plot
6. In right-hand pane, scroll down and select a *Gauge*, for example
7. Adjust *Standard options* ❭ *Min* and *Max* to `0` and `1`
8. Click *Save*, add a commit message, click *Save*

Explore Existing Dashboards

Click *Dashboards* in the left-hand pane to see all installed dashboards

For example *Kubernetes / Compute Resources / Namespace (Workloads)* shows resources used by different workloads
→ Change the *namespace* selector to select those workloads

These dashboards show off the metrics being collected from K8s

Your dashboard also shows up in this list

Persisting Dashboards

By default in this deploy, Grafana has no persistence
$\Rightarrow$ Custom dashboards will disappear when pod restarts

We could add a PVC to store changes

Better GitOps solution: Store dashboards as K8s resources

Get Dashboard Code

1. Open the dashboard in Grafana
2. In the right bar, click *Export* ❭ *Export as code*
3. Download or copy the file

Create the Resource

Grafana loads dashboards from specially configured ConfigMaps
⇒ Create `monitoring/prometheus.yaml` with the following:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-dashboard
  namespace: monitoring
  labels:
    grafana_dashboard: "1"
data:
  my-dashboard.json: |
    ...
```

Create the Resource

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-dashboard
  namespace: monitoring
  labels:
    grafana_dashboard: "1"
data:
  my-dashboard.json: |
    ...
```

Grafana will find dashboards in any namespace, by default

Create the Resource

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-dashboard
  namespace: monitoring
  labels:
    grafana_dashboard: "1"
data:
  my-dashboard.json: |
    ...
```

Grafana looks for this label to find dashboards

Create the Resource

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-dashboard
  namespace: monitoring
  labels:
    grafana_dashboard: "1"
data:
  my-dashboard.json: |
    ...
```

Paste the dashboard code here. Make sure it's properly indented.

Activity: Testing the ConfigMap

How do we:

1. Get this installed on the cluster?
2. Check that it persists?

Persisting Prometheus Database

The default deploy does not include any persistence for the Prometheus database
$\Rightarrow$ A problem that causes Prometheus to restart will delete the data we need to debug it

We will set up a PVC to persist the database.

This can be configured through the helm values file

Prometheus PVC

Create `helm/prometheus.yaml` with the following contents:

```
prometheus:
  prometheusSpec:
    retention: 15d
    retentionSize: 45GB
    storageSpec:
      volumeClaimTemplate:
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 50Gi
```

Prometheus PVC

```
prometheus:
  prometheusSpec:
    retention: 15d
    retentionSize: 45GB
    storageSpec:
      volumeClaimTemplate:
        spec:
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 50Gi
```

Rule of thumb: Make the *retentionSize* 90% of the PVC size

Upgrade the Prometheus Release

```
helm upgrade -n monitoring prometheus \
  oci://ghcr.io/prometheus-community/charts/kube-prometheus-stack \
  -f helm/prometheus.yaml
```

As before, this will take a few minutes

After Prometheus restarts, past data will have been lost

Data will not be lost on future restarts

Adding Metrics to Prometheus

Prometheus gathers metrics by hitting various HTTP endpoints
→ These must serve a particular text-based format

Many projects already expose metrics in this format

Metrics from Garage

The *garage* service exposes Prometheus metrics on port 3903

We can see them by forwarding the port:

```
kubectl port-forward -n garage service/garage 3903
```

and accessing http://localhost:3903

ServiceMonitors and PodMonitors

The K8s Prometheus Operator watches for **ServiceMonitors** and **PodMonitors** to find metrics being provided by services and pods

When these resources are created, it adds the sources to Prometheus

Service Monitor For Garage

Create `monitoring/garage.yaml`:

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: garage
  namespace: garage
  labels:
    release: prometheus
spec:
  # ...
```

—Continues→

Service Monitor For Garage

Needs to match Prometheus settings (this is the default value)

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: garage
  namespace: garage
  labels:
    release: prometheus
spec:
  # ...
```

—Continues→

Service Monitor For Garage

Look for services in this namespace

```yaml
apiVersion: monitoring.coreos.com/v1
kind: ServiceMonitor
metadata:
  name: garage
  namespace: garage
  labels:
    release: prometheus
spec:
  # ...
```

—Continues→

Service Monitor For Garage

←Continued—

```
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: garage
  endpoints:
    - targetPort: 3903
      path: /metrics
      interval: 30s
      scrapeTimeout: 10s
```

Service Monitor For Garage

←Continued—

```
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: garage
  endpoints:
    - targetPort: 3903
      path: /metrics
      interval: 30s
      scrapeTimeout: 10s
```

Look for all services in the namespace with these labels

Service Monitor For Garage

←Continued—

```
spec:
  selector:
    matchLabels:
      app.kubernetes.io/name: garage
  endpoints:
    - targetPort: 3903
      path: /metrics
      interval: 30s
      scrapeTimeout: 10s
```

Path and port where the metrics are served

Adding the Garage Metrics

Create the ServiceMonitor:

```
kubectl apply -f monitoring/garage.yaml
```

In Grafana's *Explore* page, metrics like `garage_local_disk_available` should now appear

Monitoring Container Logs

This setup only tracks numerical metrics

Container logs can be useful in understanding causes

Grafana Loki ingests and stores text logs

These can be exposed and analyzed in Grafana

Alerting

This stack provides two alerting systems:

The Prometheus **AlertManager** is configured by two CRDs:

- **PrometheusRules** define what triggers an alert
- **AlertmanagerConfigs** define how these alerts are routed

Grafana has its own copy of AlertManager embedded within it

- See the *Alterting* page
- Prometheus alerts can be used as sources here

Activity: Deploy Prometheus via ArgoCD

- Remove namespaces from the manifests
- Add monoitoring/garage.yaml to garage application
- Add prometheus application